

汇编语言程序设计

寻址方式



学习目标

- 理解“寻址方式”就是 CPU 确定操作数或有效地址的方法。
- 掌握寄存器寻址、立即数寻址，以及五种常见存储器寻址。
- 能把 C 语言中的变量、指针、数组访问对应到汇编写法。
- 会解释方括号、位移量、基址寄存器、变址寄存器和比例因子的作用。

关键词

寄存器
立即数
有效地址 EA
方括号 []
基址 / 变址 / 比例因子

最终目标

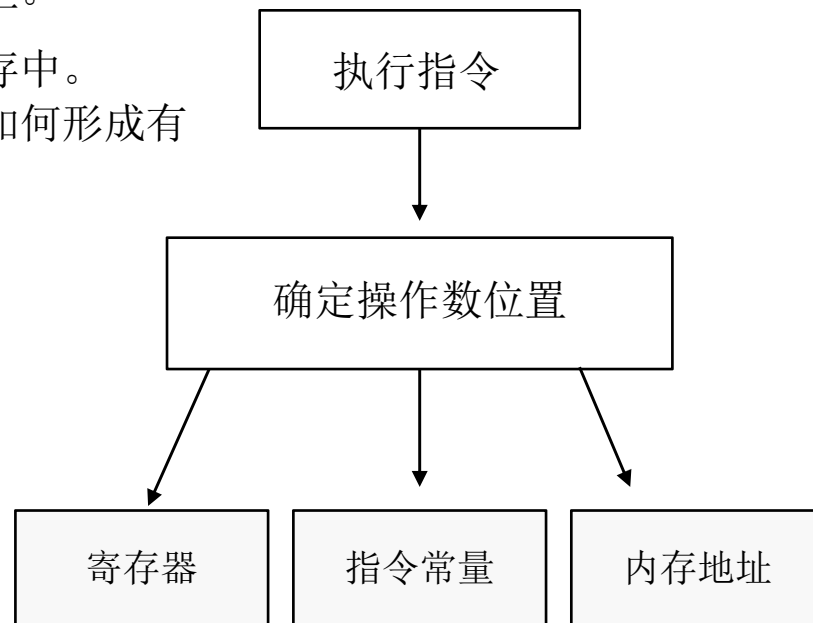
看到一条汇编指令时，能先判断：

1. 数据从哪里来？
2. 如果是内存，地址是怎样形成的？



什么是寻址方式

- 一条指令要处理操作数，必须先知道操作数在哪里。
- 操作数可能在寄存器中、在指令本身中，或在内存中。
- 因此，寻址方式的本质就是：如何找到数据，或如何形成有效地址 EA。



一句话

先找到数据，才能处理数据。

常见寻址方式总览

寻址方式	典型语句	操作数 / EA	C 场景	关键词
寄存器寻址	<code>mov eax, ebx</code>	寄存器本身	中间结果	无 []
立即数寻址	<code>mov eax, 10</code>	指令中的常量	常量赋值	常量直接给出
直接寻址	<code>mov ecx, count</code>	$EA = count$	普通变量	变量名给地址
间接寻址	<code>mov edx, [ebx]</code>	$EA = EBX$	指针 <code>*p</code>	寄存器里装地址
相对寻址	<code>mov esi, [ebx+4]</code>	$EA = EBX + 4$	<code>arr[1]</code>	基址 + 偏移
变址寻址	<code>mov eax, [ebx+edx+80h]</code>	$EA = EBX + EDX + 80H$	<code>buf[32+i]</code>	基址 + 变址 + 位移
比例变址	<code>mov eax, [ebx+esi*4-80h]</code>	$EA = EBX + ESI \times 4 - 80H$	<code>arr[i-32]</code>	自动乘元素大小

指令及其操作数

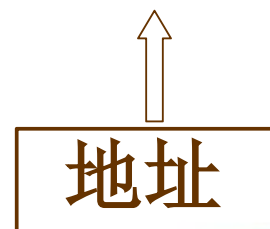
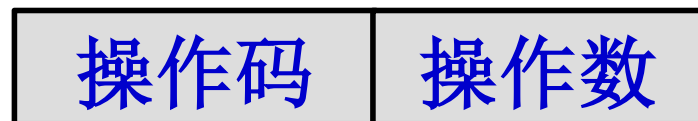
➤ 指令有两部分：操作码和操作数

▸ 操作码：处理器要执行哪种操作

不可缺少，用助记符表示

▸ 操作数：指令执行的参与者

各种操作的对象，需要通过地址指示



操作数在哪儿呢？

指令又在哪儿呢？

寻址方式 (Addressing)

➤ 通过地址访问数据或指令

➤ 数据寻址: 操作数在哪儿呢?

指令执行过程中, 访问所需要
操作的数据(操作数)

00405000H

➤ 指令寻址: 指令在哪儿呢?

一条指令执行后, 确定执行的
下一条指令的位置

00401000H

地址

数据
数据
指令
指令

操作数在那儿！

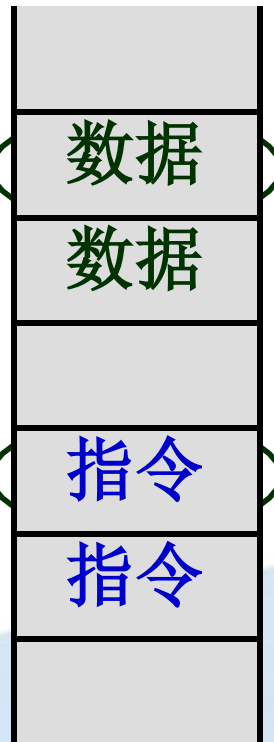
数据来自主存储器→存储器寻址

00405000H

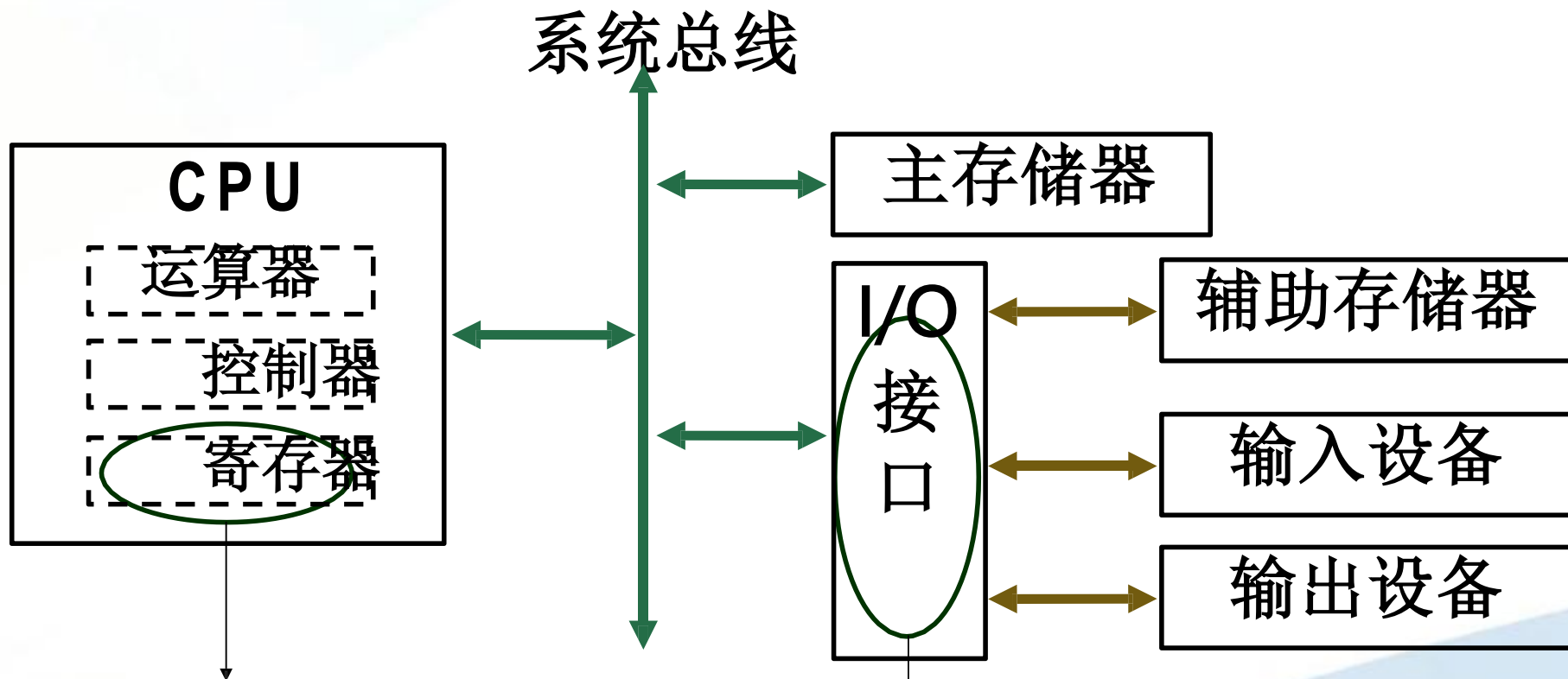
数据已经保存在指令代码中→立即数寻址

00401000H

地址



操作数在那儿！



数据来自寄存器 → 寄存器寻址

数据来自外设(接口) → 外设(I/O)寻址

数据寻址

➤立即数寻址

- ▶ 数据在指令代码中，用常量表达

操作码

操作数

➤寄存器寻址

- ▶ 数据在寄存器中，用寄存器名表示

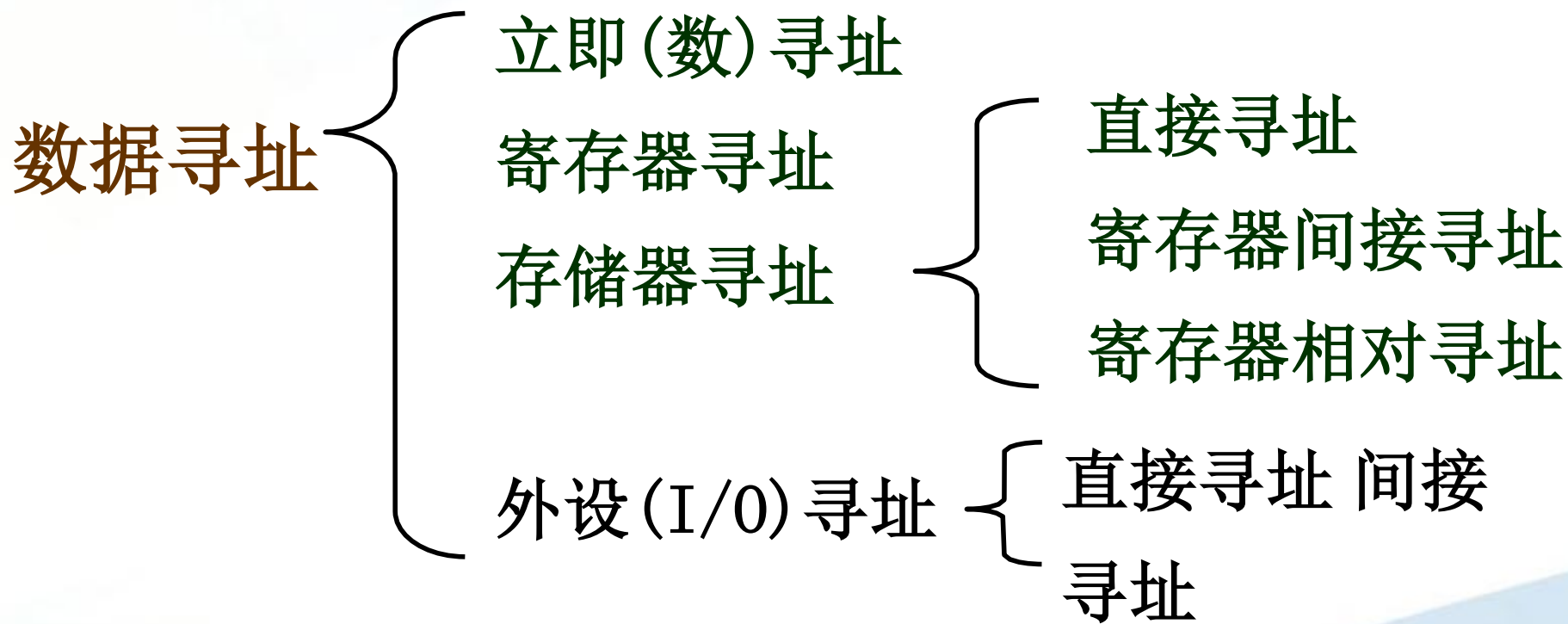
➤存储器寻址

- ▶ 数据在主存中，用存储器地址代表

➤I/O 寻址

- ▶ 数据在外设（I/O 设备）中，用I/O 地址代表

数据寻址的总结



数据寻址：通过地址查找数据（操作数）

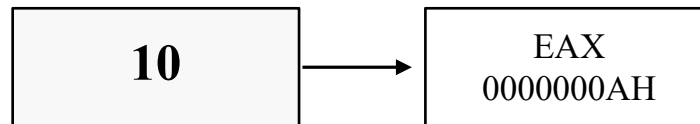
1.立即数寻址



立即数寻址

- 操作数直接写在指令代码中。
- 执行时不需要从寄存器或内存再取这个数。
- 常用于赋初值、加减常数、比较常量。

示意



mov eax, 10

关键理解： 10 是常量，不是变量，也不是地址。

对比

mov eax, 10 → 立即数
mov eax, count → 变量中的内容

立即数寻址：C / 汇编对照案例

C 语言程序

```
1 int x = 3;
2 int main() {
3     x = x + 5;
4     return 0;
5 }
```

Assembly 程序

```
1 .data
2 x DWORD 3
3
4 .code
5 main PROC
6     mov eax, x
7     add eax, 5
8     mov x, eax
9     ret
10 main ENDP
11 END main
```

对应关系

C 中的常量 5，对应汇编中的 `add eax, 5`。
这里的 5 直接写在指令中，因此属于立即数寻址。

寻址方式 (Addressing)

➤ 通过地址访问数据或指令

➤ 数据寻址: 操作数在哪儿呢?

指令执行过程中, 访问所需要

操作的数据(操作数) 指令又在哪儿呢?

➤ 指令寻址:

一条指令执行后, 确定执行的
下一条指令的位置

00405000H

00401000H

地址

数据
数据
指令
指令

操作数在那儿！

数据已经保存在指令代码中→立即数寻址

00405000H

00401000H

地址

数据

数据

指令

指令

立即数寻址

- 操作数紧跟操作码，是机器代码的一部分
 - ▶ 操作数从指令代码中得到，即立即数（Immediate）

MOV EAX,33221100H

;机器代码: **B8 00 11 22 33**

;操作码: **B8**

;立即数: **33221100**

B8	33221100
-----------	-----------------



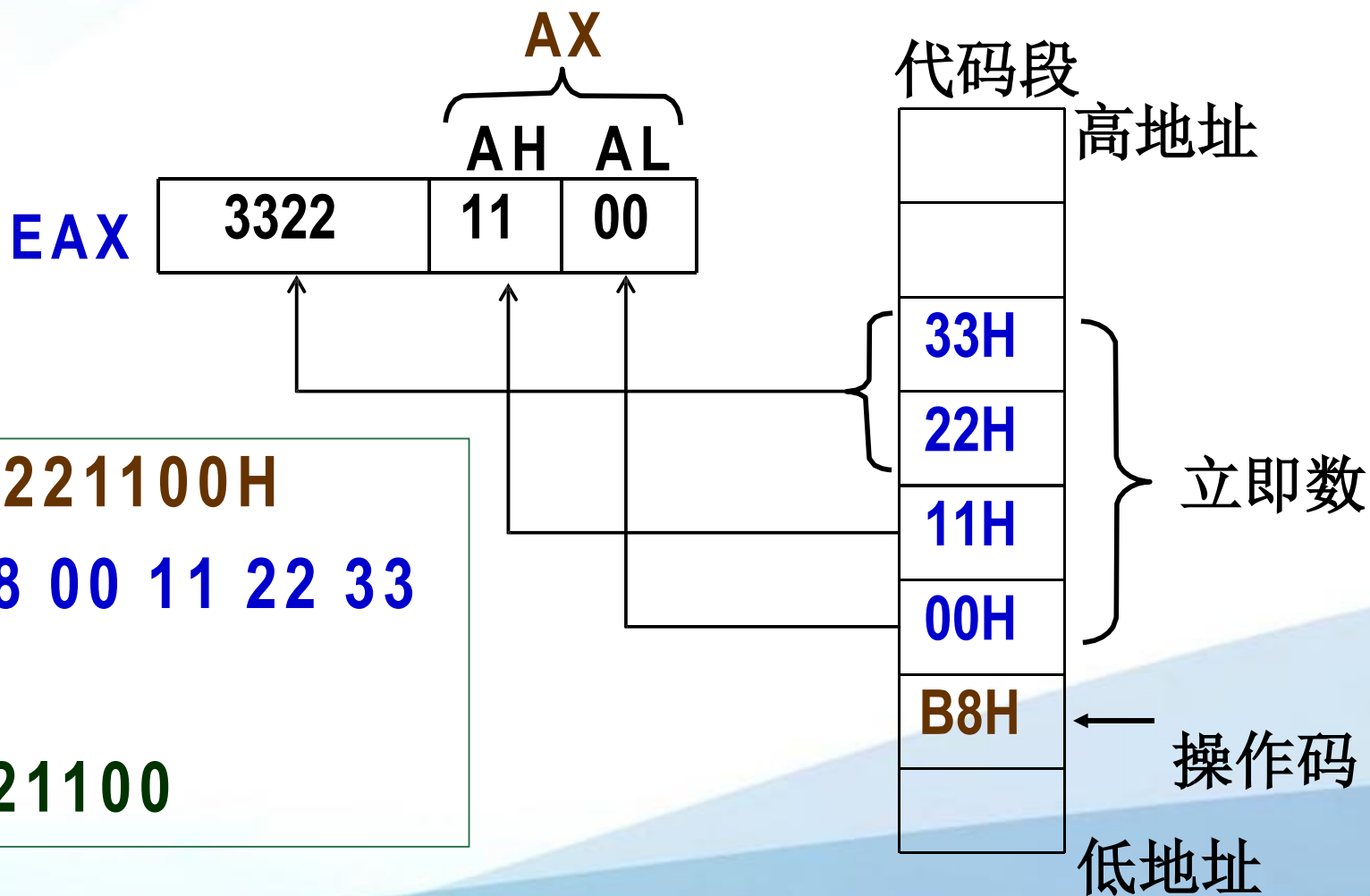
33221100

操作码	立即数
-----	-----



操作数

立即数寻址示意图



MOV EAX,33221100H

;机器代码: **B8 00 11 22 33**

;操作码: **B8**

;立即数: **33221100**

立即数寻址程序—1

;数据段

= 00000040

const = 64

00000000 87 49

bvar byte 87h,49h

00000002 12345678 0000000C

dvar dword 12345678h,12

数据段

0CH
12H
34H
56H
78H
49H
87H

dvar ==>

bvar ==>

立即数寻址程序—2

;代码段

```
00000000  B0 12  
          mov al, 12h  
  
00000002  B4 64  
          mov ah, 'd'  
  
00000004 66|BB FFFF  
labl:      mov bx, -1
```

各种立即数形式



十六进制常数

字符（ASCII码值）

十进制负数（补码）



立即数寻址程序—3

各种立即数形式



符号常量

表达式

00000008 B9 00000040

mov ecx, const

0000000D BA 00000040

mov edx, const*4/type dvar

const = 64

bvar byte 87h,49h

dvar dword 12345678h,12

立即数寻址程序—4

00000012 BE 00000000 R

mov esi, offset bvar

00000017 BF 00000004 R

mov edi, labl

labl: mov bx, -1

各种立即数形式



变量的偏移地址

标号的偏移地址

代码段使用标号名代表其偏移地址



立即数寻址程序—5

0000001C C6 05 00000000 R 4C

mov bvar, 01001100b

00000023 C7 05 00000006 R 00000012

mov dvar+4, 12h

立即数本身没有类型



字节变量bvar类型

双字变量dvar类型

立即数的类型由对应的寄存器或变量类型决定

立即数寻址：C / 汇编对照案例

C 语言程序

```
1 int x = 3;
2 int main() {
3     x = x + 5;
4     return 0;
5 }
```

Assembly 程序

```
1 .data
2 x DWORD 3
3
4 .code
5 main PROC
6     mov eax, x
7     add eax, 5
8     mov x, eax
9     ret
10 main ENDP
11 END main
```

对应关系

C 中的常量 5，对应汇编中的 `add eax, 5`。
这里的 5 直接写在指令中，因此属于立即数寻址。



立即数寻址小结

- 立即数是直接写在指令里的常量。
- 适合赋值、加减和比较。
- 不需要额外的地址查找过程。

区分

`mov eax, 5` 与 `mov eax, x`
前者取常量，后者取变量内容。

常见形式

```
mov ebx, 10
add eax, 5
cmp ecx, 0
```


本讲总结

➤立即数寻址

- ▶ 操作数紧跟操作码，是机器代码的一部分
- ▶ 操作数从指令代码中得到，即立即数（Immediate）

➤立即数寻址方式只用于源操作数

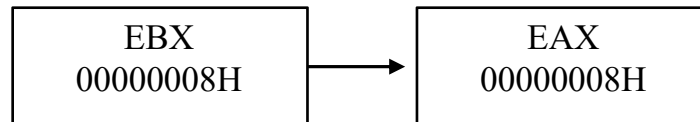
- ▶ 常用来给寄存器和存储单元赋值
- ▶ 用常量形式直接表达

符号	含义
i8	8位立即数
i16	16位立即数
i32	32位立即数
imm	立即数

2. 寄存器寻址

- 操作数直接放在寄存器中。
- 指令直接对寄存器里的数据进行操作，不需要访问内存。
- 常用于运算过程中的中间结果传送。

示意



mov eax, ebx

含义：把 EBX 中的数据送入 EAX

识别方法

没有 [], 操作数本身就是寄存器。



寄存器寻址：C / 汇编对照案例

C 语言程序

```
1 int a = 5;
2 int b = 8;
3 int c;
4 int main() {
5     c = a + b;
6     return 0;
7 }
```

Assembly 程序

```
1 .data
2 a DWORD 5
3 b DWORD 8
4 c DWORD ?
5
6 .code
7 main PROC
8     mov eax, a
9     mov ebx, b
10    mov ecx, eax
11    add ecx, ebx
12    mov c, ecx
13    ret
14 main ENDP
15 END main
```

对应关系

关键寄存器寻址语句是 `mov ecx, eax`。

它表示把一个寄存器中的中间结果传送到另一个寄存器。

寻址方式 (Addressing)

➤ 通过地址访问数据或指令

➤ 数据寻址: **操作数在哪儿呢?**

指令执行过程中, 访问所需要
操作的数据(操作数)

00405000H

➤ 指令寻址: **指令又在哪儿呢?**

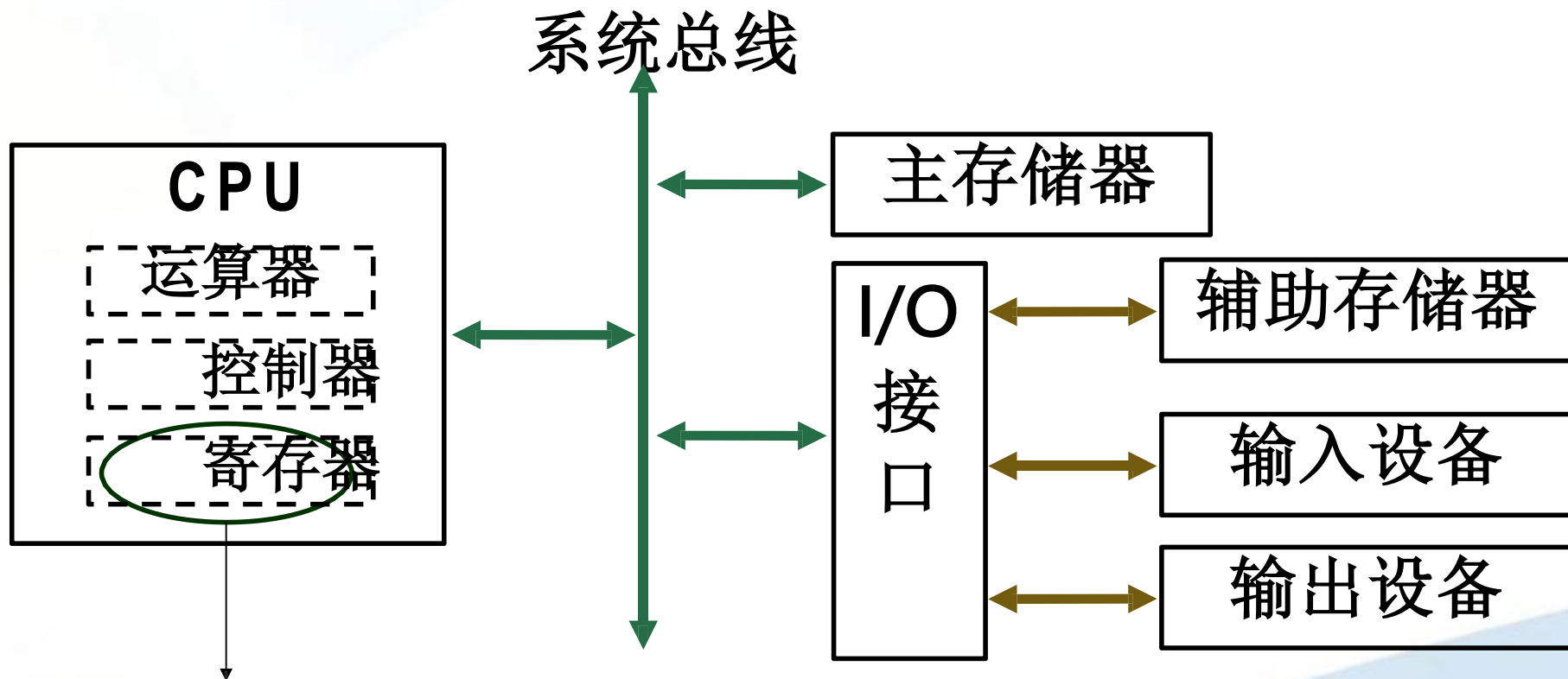
一条指令执行后, 确定执行
的下一条指令的位置

00401000H

地址

数据
数据
指令
指令

操作数在那儿！



数据来自寄存器 → 寄存器寻址

寄存器寻址

➤ 操作数存放在处理器的内部寄存器中

▸ 用寄存器名表示它的内容

MOV EBX, EAX

;目的操作数和源操作数

;均采用寄存器寻址

EBX ← **EAX**

操作码

寄存器地址

寄存器

操作数

32位通用寄存器: **EAX EBX ECX EDX ESI EDI EBP ESP**

16位通用寄存器: **AX BX CX DX SI DI BP SP**

8位通用寄存器: **AH AL BH BL CH CL DH DL**

寄存器寻址程序

;代码段

```
mov al, ah  
mov bx, ax  
mov ebx, eax  
mov dx, ds  
mov es, dx
```

源操作数和目的操作数
均是寄存器寻址

;代码段

```
mov al, 12  
mov bx, 12  
mov bvar, cl  
mov wvar, dx  
mov dvar, edx
```

目的操作数是寄存器寻址

源操作数是寄存器寻址

8位寄存器的类型是字节型
16位寄存器的类型是字型
32位寄存器的类型是双字型

语法错

`mov edi,si`

eg0209.asm(11) : error A2022:

instruction operands must be the same size

出错了!

错误语句的行号

错误编号

错误信息

➤ 常见语法错误原因

- ▶ 拼写错误、多余的空格、遗忘的后缀字母或前导0、不正确的标点、太过复杂的常量或表达式
- ▶ 操作数类型不匹配、错用寄存器



寄存器寻址：C / 汇编对照案例

C 语言程序

```
1 int a = 5;
2 int b = 8;
3 int c;
4 int main() {
5     c = a + b;
6     return 0;
7 }
```

Assembly 程序

```
1 .data
2 a DWORD 5
3 b DWORD 8
4 c DWORD ?
5
6 .code
7 main PROC
8     mov eax, a
9     mov ebx, b
10    mov ecx, eax
11    add ecx, ebx
12    mov c, ecx
13    ret
14 main ENDP
15 END main
```

对应关系

关键寄存器寻址语句是 `mov ecx, eax`。

它表示把一个寄存器中的中间结果传送到另一个寄存器。



寄存器寻址小结

- 操作数在寄存器中。
- 不需要访问内存，因此速度快。
- 常见于运算中间过程、函数调用约定和临时结果保存。

易错点

`mov eax, ebx` 是传送寄存器中的数据，不是按照 `EBX` 的值去访问内存。

典型识别

```
add eax, ebx
sub ecx, edx
mov esi, eax
```

本讲总结

➤寄存器寻址

- ▶ 操作数存放在处理器的内部寄存器中
- ▶ 用寄存器名表示它的内容

➤寄存器寻址方式简单快捷，最常使用

- ▶ 绝大多数指令采用通用寄存器
- ▶ 部分指令支持专用寄存器
(例如段寄存器)

符号	含义
r8	8位寄存器
r16	16位寄存器
r32	32位寄存器
reg	通用寄存器

3. 存储器寻址

寻址方式 (Addressing)

➤ 通过地址访问数据或指令

➤ 数据寻址: **操作数在哪儿呢?**

指令执行过程中, 访问所需要
操作的数据(操作数)

00405000H

➤ 指令寻址: 一条指令执行后,
确定执行的下一条指令的位置
指令又在哪儿呢

?

00401000H

地址

数据
数据
指令
指令

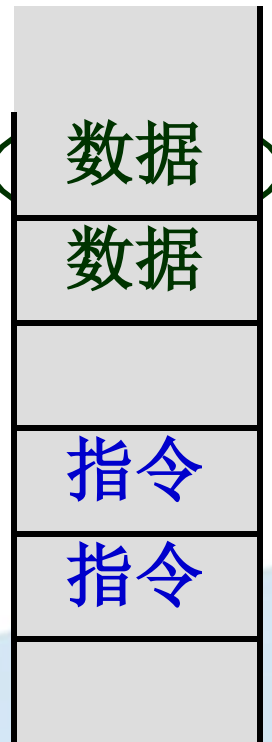
操作数在那儿！

数据来自主存储器→存储器寻址

00405000H

00401000H

地址

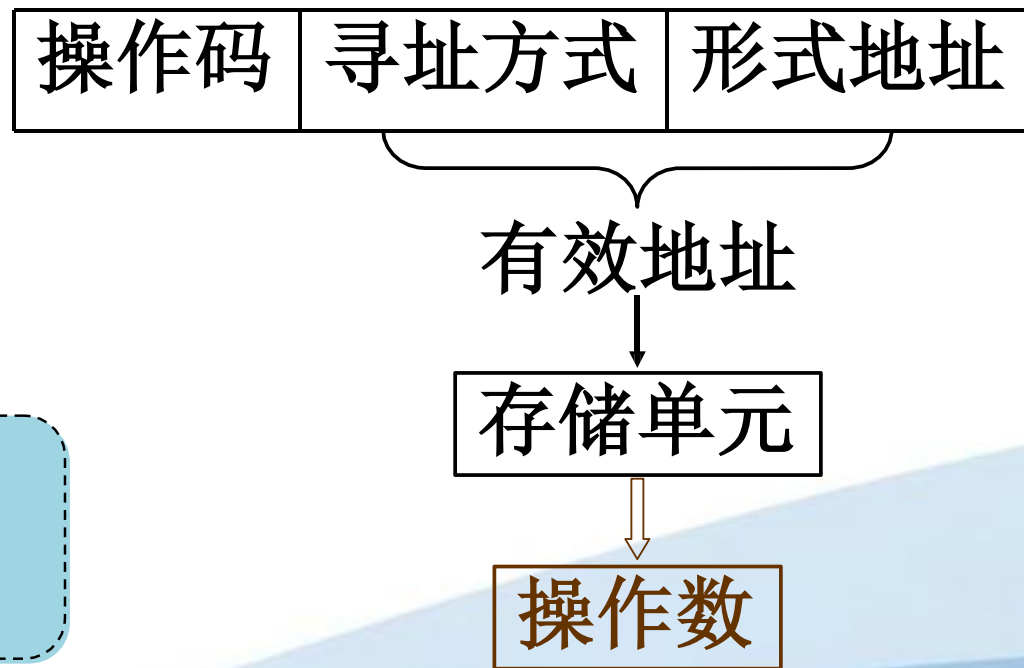


存储器寻址

➤ 操作数在主存中，通过存储器地址指示

- ▶ 指令代码表达形式地址
- ▶ 由形式地址结合规则 经过计算得到有效地址EA (Effective Address)

处理器将有效地址转换为物理地址访问存储单元



存储器的逻辑地址

- 编程时，存储器地址使用逻辑地址
 - ▶ 段寄存器指示段基地址
 - 绝大部分情况使用默认规定，无需表达
 - 有时需要显式说明（称为段超越）
 - ▶ 偏移地址由各种存储器寻址方式计算
 - 这被称为有效地址EA

逻辑地址 = 段基地址 : 偏移地址

段寄存器指示段基地址

- 绝大部分情况使用默认规定，无需表达
 - ▶ 读取指令，一定是代码段**CS**
 - ▶ 堆栈操作，针对堆栈段**SS**
 - ▶ 读写数据，默认在数据段**DS**
 - 有时需要显式说明（称为段超越）
 - ▶ 使用段超越指令前缀（段寄存器名后跟冒号）

CS: DS: SS: ES: FS: GS:

段寄存器的默认和超越

访问存储器的方式	默认	可超越	偏移地址
读取指令	CS	无	EIP
堆栈操作	SS	无	ESP
一般数据访问(下列除外)	DS	CS ES SS FS GS	有效地址 EA
EBP/ESP 基址的数据访问	SS	CS ES DS FS GS	有效地址 EA
串指令的源操作数	DS	CS ES SS FS GS	ESI
串指令的目的操作数	ES	无	EDI

32位有效地址的组成

基址寄存器 + 变址寄存器 × 比例 + 位移量

任何8个32位通用寄存器之一

1, 2, 4或8

除ESP外的任何32位通用寄存器之一

$EBX + ESI * 4 + 80h$

8或32位有符号值

16位有效地址的组成

基址寄存器 + 变址寄存器 + 位移量



$$BX + SI + 80h$$

32位存储器寻址

基址寄存器 + 变址寄存器 × 比例 + 位移量

$$\left\{ \begin{array}{c} \text{无} \\ \text{EAX} \\ \text{EBX} \\ \text{ECX} \\ \text{EDX} \\ \text{ESI} \\ \text{EDI} \\ \text{EBP} \\ \text{ESP} \end{array} \right\} + \left\{ \begin{array}{c} \text{无} \\ \text{EAX} \\ \text{EBX} \\ \text{ECX} \\ \text{EDX} \\ \text{ESI} \\ \text{EDI} \\ \text{EBP} \end{array} \right\} \times \left\{ \begin{array}{c} 1 \\ 2 \\ 4 \\ 8 \end{array} \right\} + \left\{ \begin{array}{c} \text{8位位移量} \\ \text{32位位移量} \end{array} \right\}$$

组合成多种存储器寻址方式

本讲总结

➤ 存储器寻址

- ▶ 操作数在主存中，通过存储器地址指示
- ▶ 指令代码表达形式地址

➤ 形式地址结合规则计算得到有效地址EA

有效地址EA

= 基址寄存器

+ 变址寄存器 × 比例

+ 位移量

符号	含义
m8	8位存储器单元
m16	16位存储器单元
m32	32位存储器单元
mem	存储器操作数

存储器的寻址方式

1. 存储器的直接寻址：有效地址只有位移偏移量部分，且只包含在指令代码中，就是存储器的直接寻址方式。直接寻址用于存取变量。例如，将变量COUNT的内容传给ECX的指令：

`mov ecx, count`;也可以表达`mov ecx[count]`

2. 寄存器间接寻址：有效地址放在寄存器中。例如：

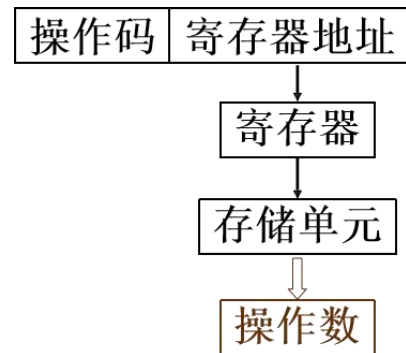
`mov edx, [ebx]`;双字传送，EBX间接寻址主存数据段；

如果EBX=405000H, 那么该指令等价于 `mov edx, DS:[405000H]`

3. 寄存器相对寻址：寄存器相对寻址的有效地址是寄存器的内容与位移量之和。例如：

`mov esi, [ebx+4]`;源操作数可以表达为 `[4][ebx]`, 或者`4[ebx]`

偏移量还可以是其他常量，以及变量所表示的地址，例如 `mov eax, count[esi]`



三种寻址方式总览

寻址方式	典型指令	有效地址	C 场景	关键词
直接寻址	<code>mov ecx, count</code>	$EA = \text{位移量}$	<code>x = count;</code>	变量名固定
寄存器间接寻址	<code>mov edx, [ebx]</code>	$EA = EBX$	<code>x = *p;</code>	寄存器里装地址
寄存器相对寻址	<code>mov esi, [ebx+4]</code>	$EA = EBX + 4$	<code>x = arr[1];</code>	基址 + 偏移

核心结论：区别不在 `mov` 指令，而在“内存地址是怎样得到的”。

3.1 存储器的直接寻址

存储器的寻址方式

1. 存储器的直接寻址：有效地址只有位移偏移量部分，且只包含在指令代码中，就是存储器的直接寻址方式。直接寻址用于存取变量。例如，将变量COUNT的内容传给ECX的指令：

`mov ecx, count`;也可以表达`mov ecx, [count]`

2. 寄存器间接寻址：有效地址放在寄存器中。例如：

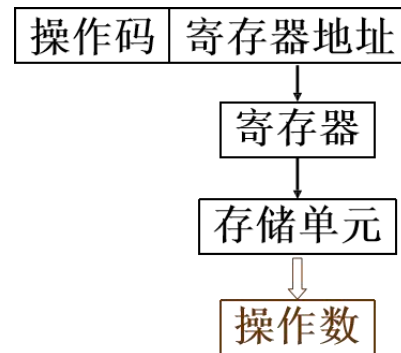
`mov edx, [ebx]`;双字传送，EBX间接寻址主存数据段；

如果EBX=405000H，那么该指令等价于 `mov edx, DS:[405000H]`

3. 寄存器相对寻址：寄存器相对寻址的有效地址是寄存器的内容与位移量之和。例如：

`mov esi, [ebx+4]`;源操作数可以表达为 `[4][ebx]`，或者`4[ebx]`

偏移量还可以是其他常量，以及变量所表示的地址，例如 `mov eax, count[esi]`



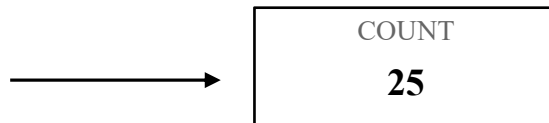


存储器的直接寻址

- 有效地址只有位移偏移量部分，地址信息已经包含在指令代码中。

- 适合访问数据段中位置固定的变量。
`mov ecx, count`

地址示意



指令直接定位到变量 COUNT

为什么要这样写？

- 当程序已经知道变量名时，不需要再借助寄存器保存地址。
- 因此访问变量 COUNT 时，直接在指令中写出变量名最自然。



直接寻址案例：普通变量赋值

C 语言程序

```
1 int count = 25;
2 int main() {
3     int x;
4     x = count;
5     return 0;
6 }
```

Assembly 程序

```
1 .data
2 count DWORD 25
3 x      DWORD ?
4
5 .code
6 main PROC
7     mov ecx, count
8     mov x, ecx
9     ret
10 main ENDP
11 END main
```

对应关系: `x = count;` → 先执行 `mov ecx, count` 取出变量值, 再执行 `mov x, ecx` 完成保存。

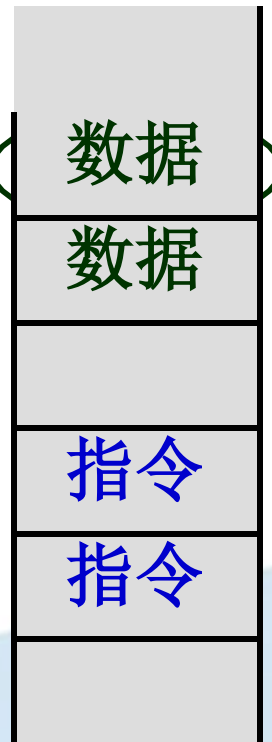
操作数在那儿！

数据来自主存储器→存储器寻址

00405000H

00401000H

地址

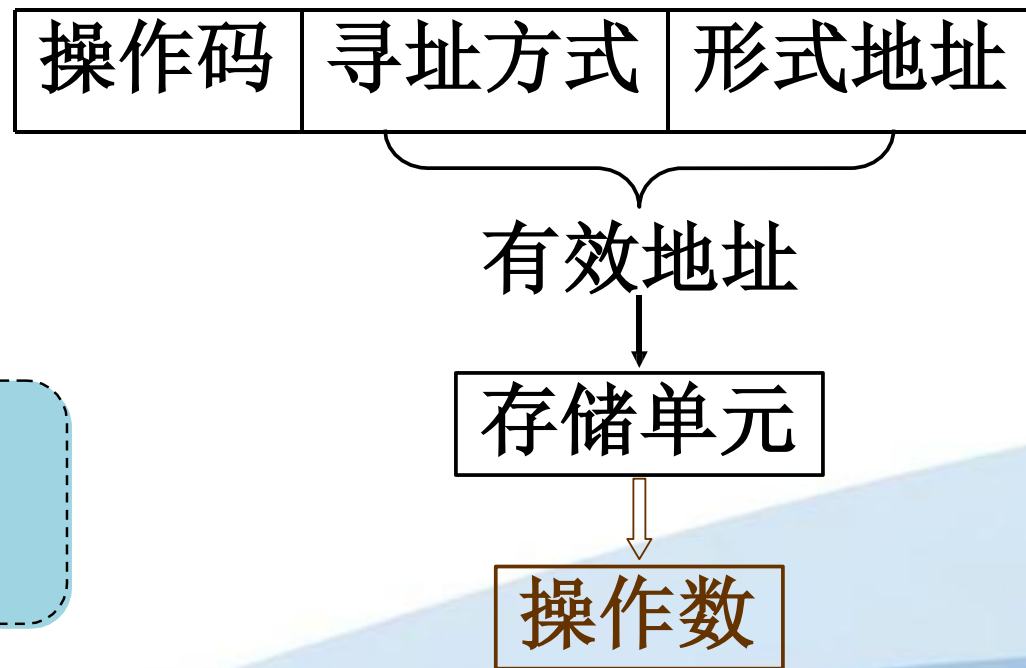


存储器寻址

➤ 操作数在主存中，通过存储器地址指示

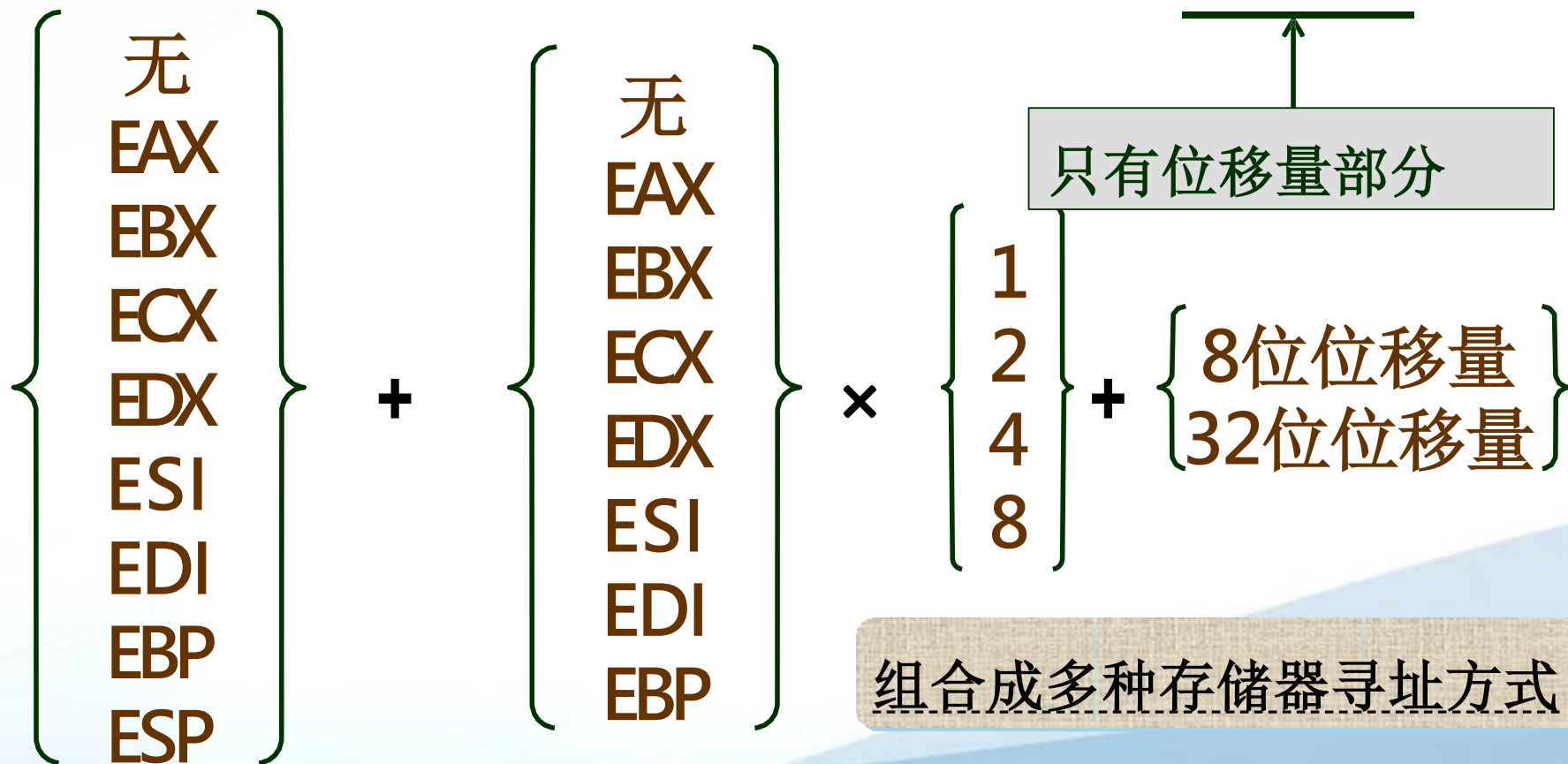
- ▶ 指令代码表达形式地址
- ▶ 由形式地址结合规则 经过计算得到有效地址EA
(Effective Address)

为物理地址访问存储单元



32位存储器寻址

基址寄存器 + 变址寄存器 × 比例 + 位移量



直接寻址

- 有效地址只有位移量部分，直接包含在指令代码中
 - ▶ 用变量名（或加中括号）表示偏移地址

MOV ECX,COUNT

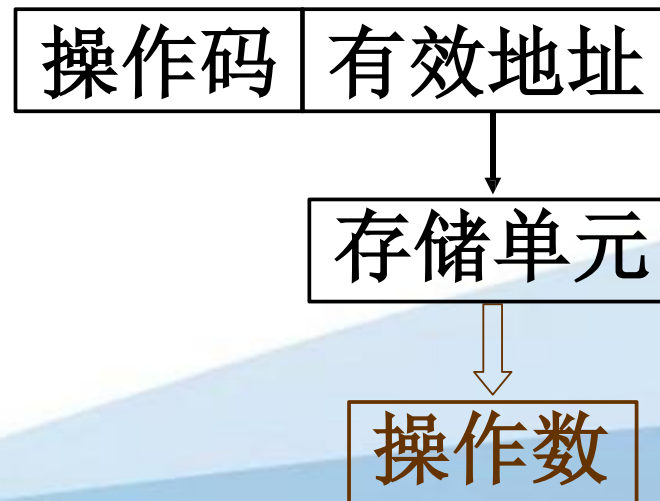
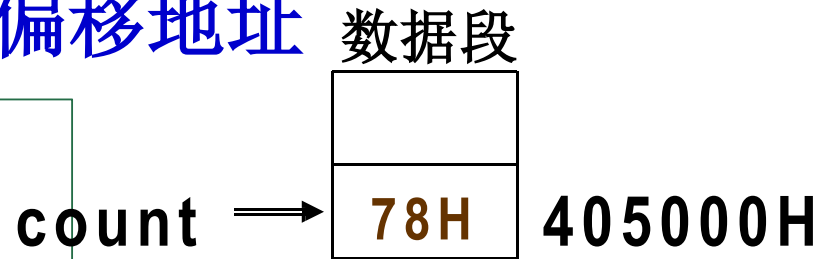
MOV ECX,[COUNT]

MOV ECX, DS:[405000H]

;指令代码: 8B 0D 00 50 40 00

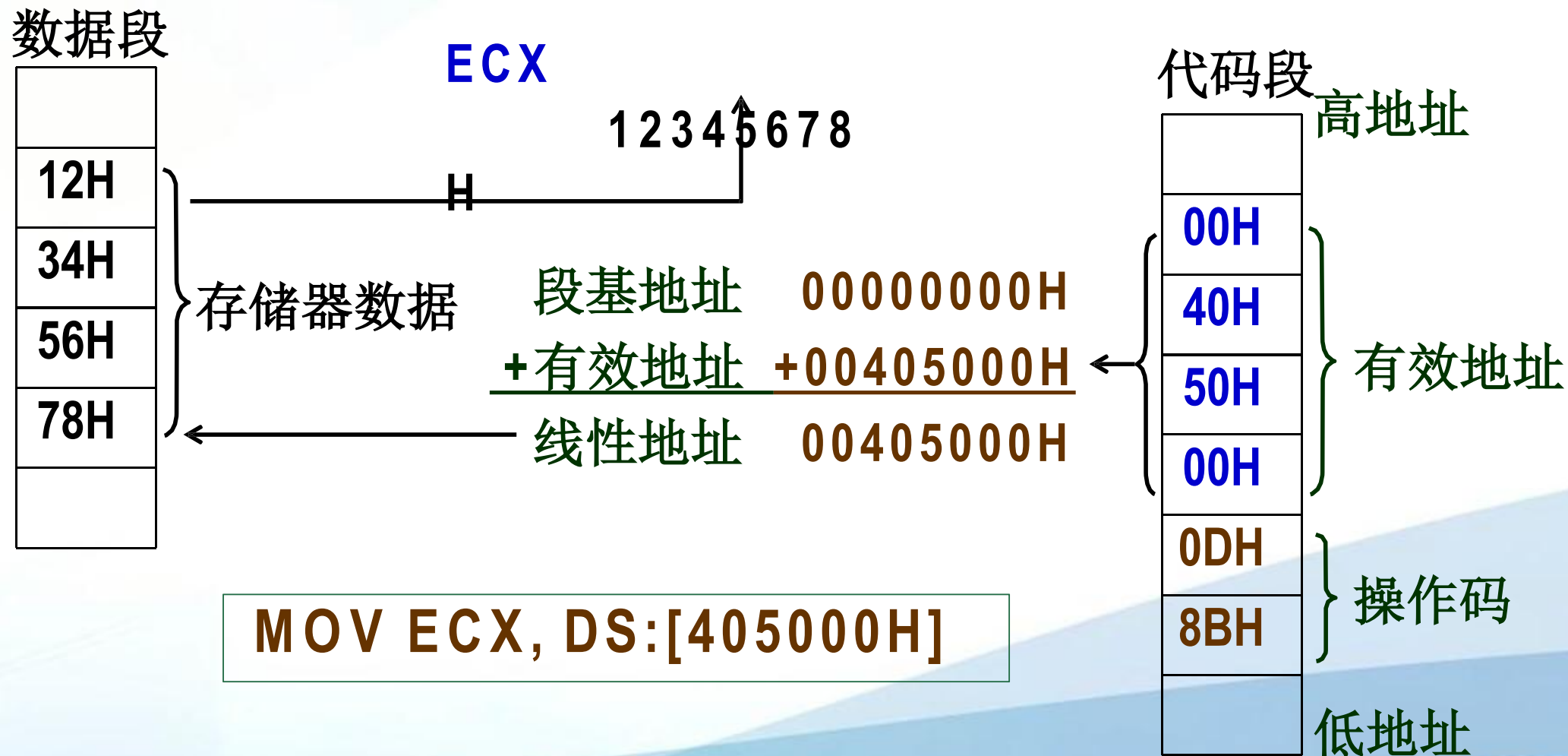
;操作码: 8B 0D

;操作数: 有效地址 00405000H



COUNT 变量所在的地址 = 405000H, 那么该指令等价于 mov ecx, DS:[405000H]

存储器直接寻址示意图



存储器直接寻址程序—1

;数据段

00000000 87 49

bvar byte 87h,49h

00000002 12345678 0000000C

dvar dword 12345678h,12

数据段

0CH
12H
34H
56H
78H
49H
87H

dvar ==> 00000002

bvar ==> 00000000

存储器直接寻址程序—2

;代码段

00000000 8A 0D 00000000 R

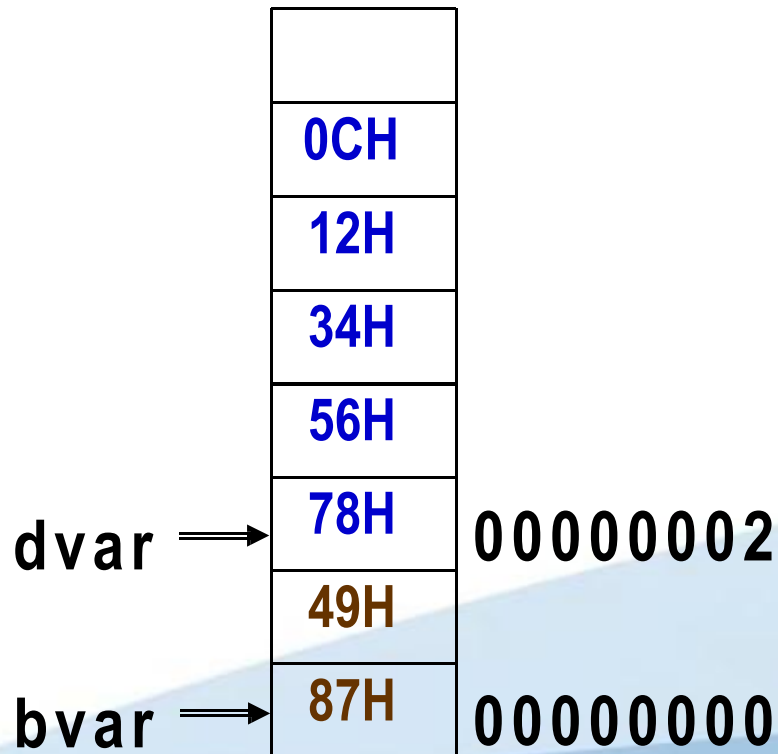
mov cl, bvar

00000006 8B 15 00000002 R

mov edx, dvar

主存操作数常通过变量形式引用 一般
不需要使用段超越前缀指令

数据段



存储器直接寻址程序—3

0000000C 88 35 00000001 R

mov bvar+1,dh

00000012 66|89 15 00000004 R

mov word ptr dvar+2,dx

00000019 C7 05 00000002 R 87654321

mov dvar,87654321h

源操作数和目的操作数均可以 采用
存储器操作数，但不能同时使用

数据段

0CH
12H
34H
56H
78H
49H
87H

dvar ==> 00000002

bvar ==> 00000000

存储器直接寻址程序—4

`mov dvar+4,dvar`

出错了!

eg0210.asm(13) : error A2070:

invalid instruction operands

错误语句的行号

错误编号

错误信息

➤ 常见语法错误原因

- ▶ 拼写错误、多余的空格、遗忘的后缀字母或前导0、不正确的标点、太过复杂的常量或表达式
- ▶ 操作数类型不匹配、2个存储器操作数



直接寻址案例：普通变量赋值

C 语言程序

```
1 int count = 25;
2 int main() {
3     int x;
4     x = count;
5     return 0;
6 }
```

Assembly 程序

```
1 .data
2 count DWORD 25
3 x      DWORD ?
4
5 .code
6 main PROC
7     mov ecx, count
8     mov x, ecx
9     ret
10 main ENDP
11 END main
```

对应关系: `x = count;` → 先执行 `mov ecx, count` 取出变量值, 再执行 `mov x, ecx` 完成保存。

本讲总结

- 存储器的直接寻址
 - 指令代码中直接给出有效地址
- 直接寻址常用于存取变量
 - 直接使用变量名表达：变量名
 - 变量名加或减一个常量：变量名+n 变量名-n
 - 或者用中括号括起变量名：[变量名] 变量名[n]
- 直接寻址的操作数具有变量定义的类型
 - 有时需要通过PTR操作符进行类型转换

3.2 存储器的寄存器间接寻址

存储器的寻址方式

1. 存储器的直接寻址：有效地址只有位移偏移量部分，且只包含在指令代码中，就是存储器的直接寻址方式。直接寻址用于存取变量。例如，将变量COUNT的内容传给ECX的指令：

`mov ecx, count`;也可以表达`mov ecx[count]`

2. 寄存器间接寻址：有效地址放在寄存器中。例如：

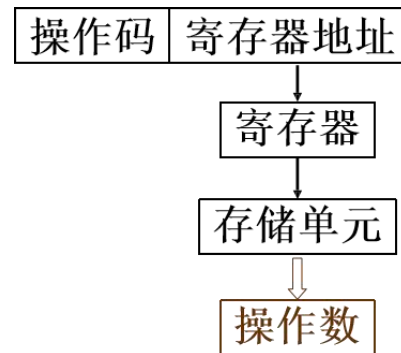
`mov edx, [ebx]`;双字传送，EBX间接寻址主存数据段；

如果EBX=405000H, 那么该指令等价于 `mov edx, DS:[405000H]`

3. 寄存器相对寻址：寄存器相对寻址的有效地址是寄存器的内容与位移量之和。例如：

`mov esi, [ebx+4]`;源操作数可以表达为 `[4][ebx]`, 或者`4[ebx]`

偏移量还可以是其他常量，以及变量所表示的地址，例如 `mov eax, count[esi]`

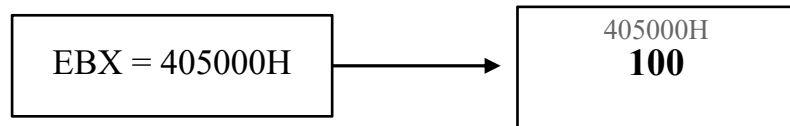




寄存器间接寻址

- 有效地址放在寄存器中，方括号表示“到该地址对应的内存单元取内容”。
- 典型指令：`mov edx, [ebx]`
- 若 `EBX = 405000H`，则该指令等价于 `mov edx, DS:[405000H]`

两层含义



`mov edx, [ebx]` 读取的是 100

关键区分：

`mov edx, ebx` → 取地址

`mov edx, [ebx]` → 取地址中的内容

寄存器间接寻址案例：指针解引用

C 语言程序

```
1 int value = 100;
2 int *p = &value;
3 int main() {
4     int x;
5     x = *p;
6     return 0;
7 }
```

Assembly 程序

```
1 .data
2 value DWORD 100
3 p      DWORD ?
4 x      DWORD ?
5
6 .code
7 main PROC
8     mov ebx, OFFSET value
9     mov p, ebx
10    mov ebx, p
11    mov edx, [ebx]
12    mov x, edx
13    ret
14 main ENDP
15
```

理解：p 中保存地址，*p 表示根据地址取值；因此必须出现 `mov edx, [ebx]`。

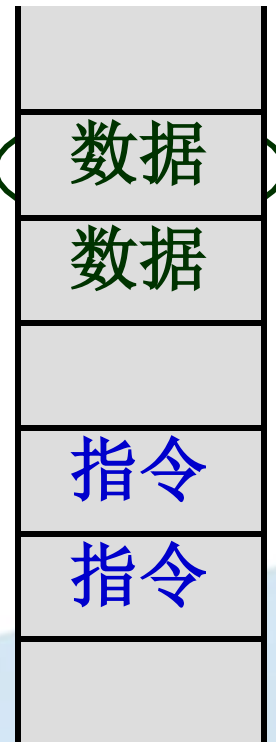
操作数在那儿！

数据来自主存储器→存储器寻址

00405000H

00401000H

地址

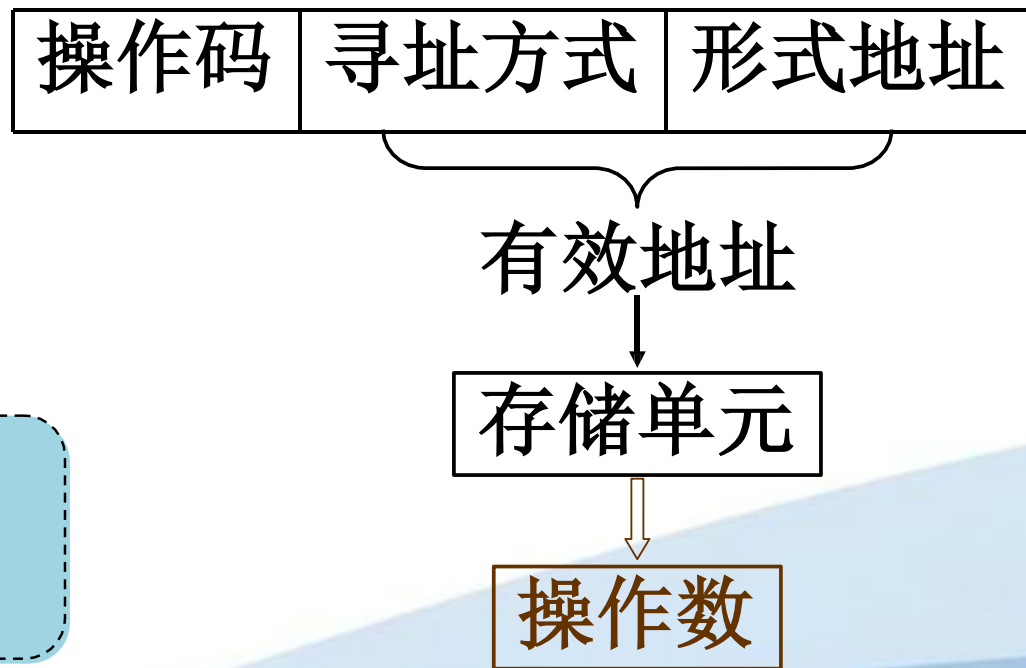


存储器寻址

➤ 操作数在主存中，通过存储器地址指示

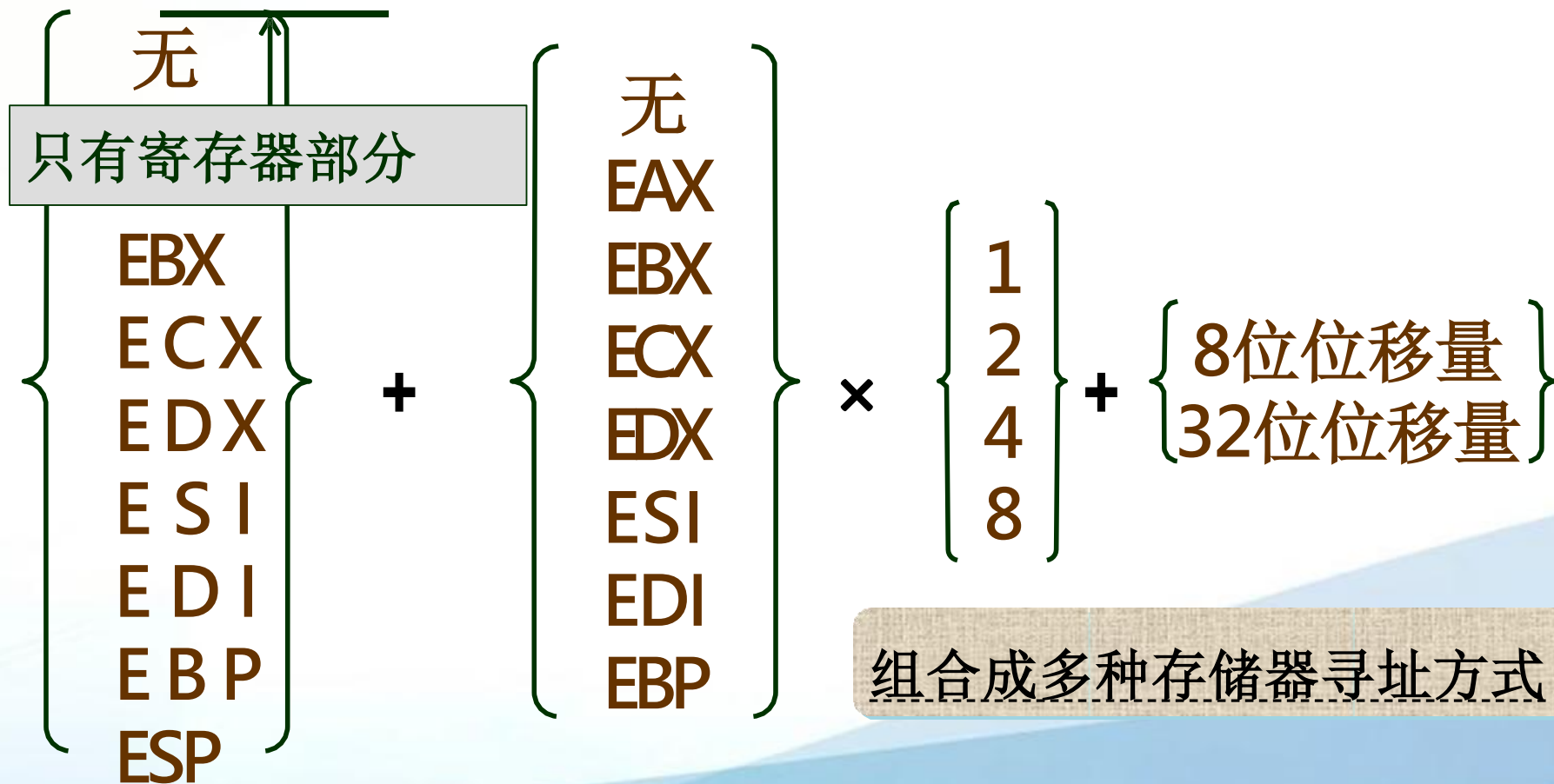
- ▶ 指令代码表达形式地址
- ▶ 由形式地址结合规则 经过计算得到有效地址EA (Effective Address)

处理器将有效地址转换为物理地址访问存储单元



32位存储器寻址

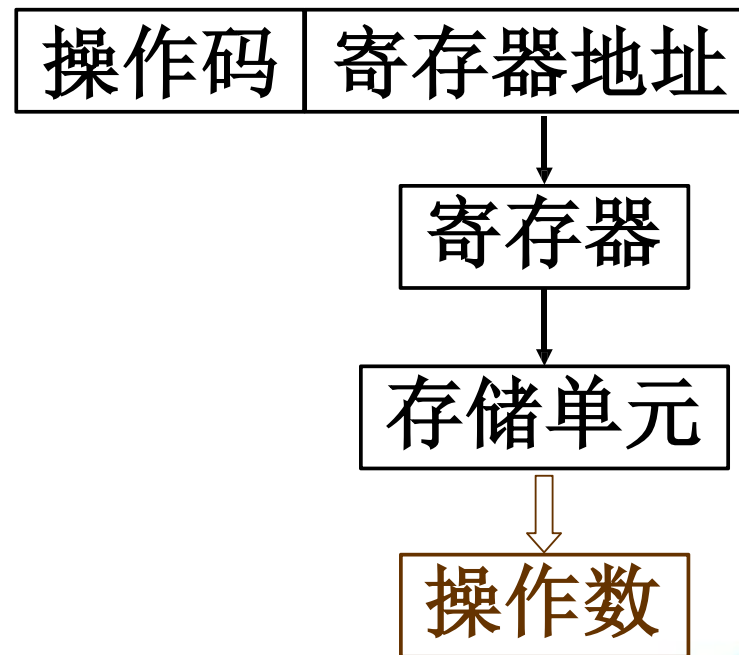
基址寄存器 + 变址寄存器 × 比例 + 位移量



寄存器间接寻址

- 有效地址存放在寄存器中
(寄存器内容 = 偏移地址 = 有效地址)
 - ▶ 用中括号括起寄存器表达

<code>mov edx, [ebx]</code>	; 双字量传送
<code>mov cx, [esi]</code>	; 字量传送
<code>mov [edi], al</code>	; 字节量传送



寄存器间接寻址的数据 由另一个操作数的寄存器或变量类型决定

寄存器间接寻址未说明数据类型

MOV [EBX],100 ;错误, 数据类型不明确

mov byte ptr [ebx],100

;正确: byte ptr说明是字节操作

mov word ptr [ebx],100

;正确: word ptr说明是字操作

mov dword ptr [ebx],100

;正确: dword ptr说明是双字操作

立即数也无类型 故
需要显式说明

字符串复制

;数据段

srcmsg byte 'Try your best, why not.',0

dstmsg byte sizeof srcmsg dup(?)

srcmsg

'T'	'r'	'y'		'n'	'o'	't'	'.'	0
-----	-----	-----	--	-----	-----	-----	-----	---

dstmsg

--	--	--	--	--	--	--	--	--

寄存器指向字符

```
mov ecx,lengthof srcmsg
```

```
mov esi,offset srcmsg
```

```
mov edi,offset dstmsg
```

ECX = 字符个数

srcmsg

'T'	'r'	'y'		'n'	'o'	't'	'.'	0
-----	-----	-----	--	-----	-----	-----	-----	---

ESI

dstmsg

--	--	--	--	--	--	--	--	--

EDI

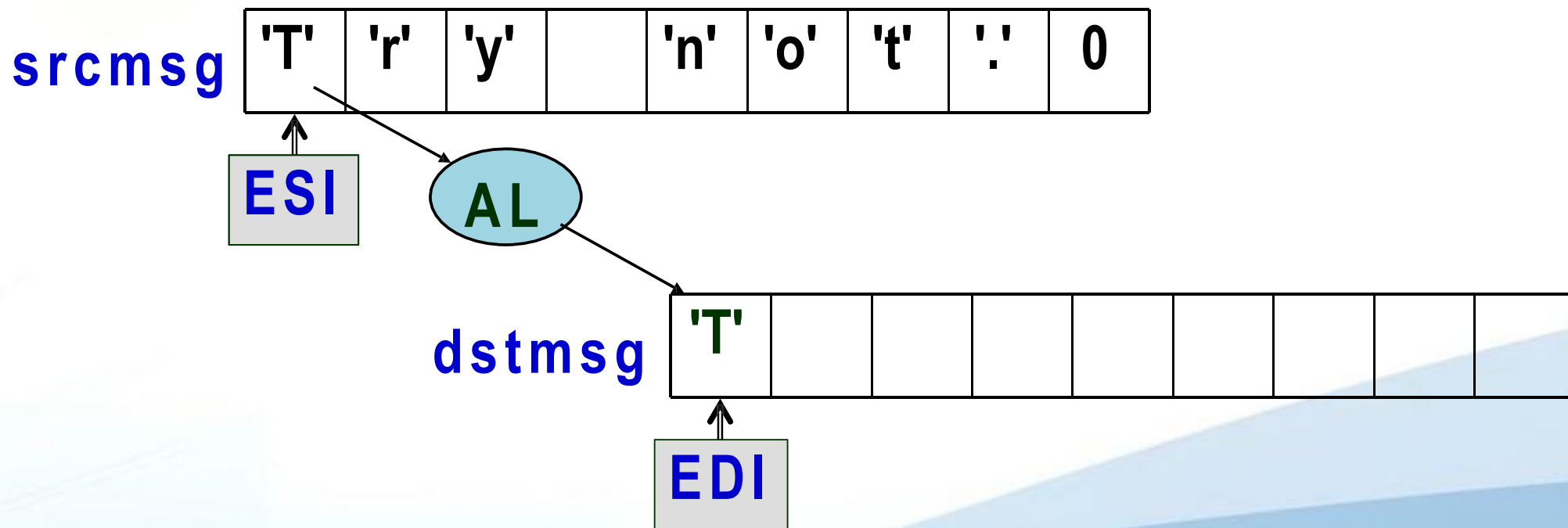
字符传送

mov al, [esi]

;取源串一个字符送AL

mov [edi], al

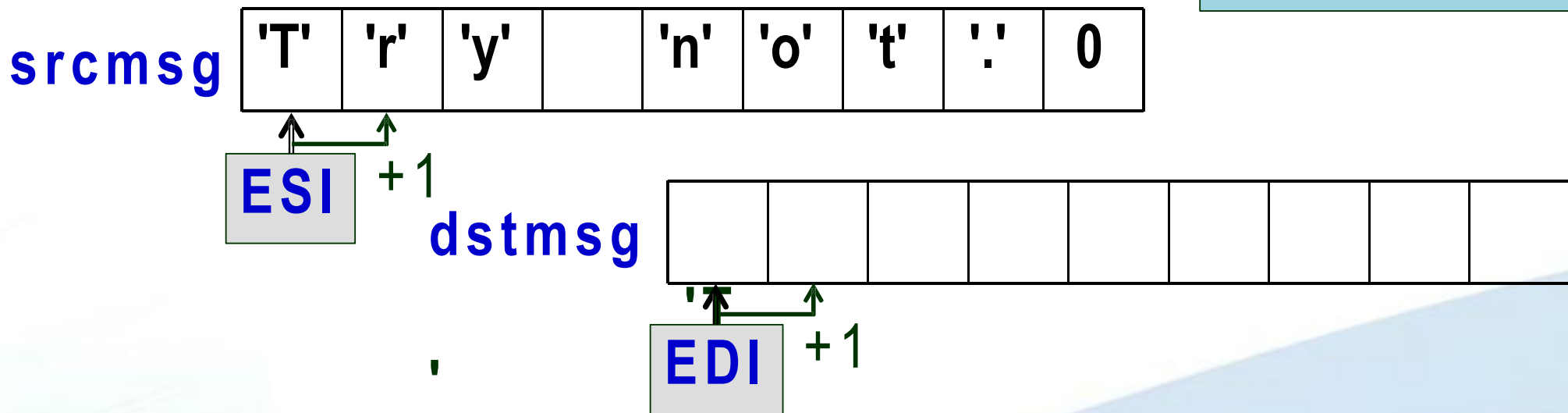
;将AL传送给目的串位置



指向下一个字符

add esi,1 ;源串指针加1
add edi,1 ;目的串指针加1

ADD是加法指令



字符串复制：逐个字符传送

again: **mov al, [esi]**

;传送一个字符

;指向下一个字符

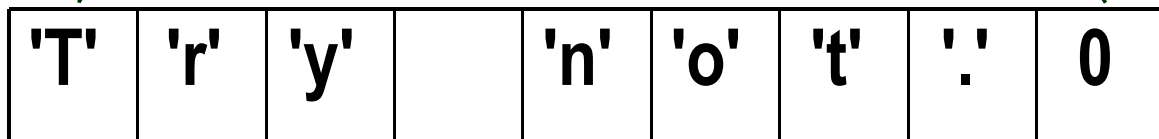
loop again

;字符个数减1，不为0继续

LOOP是循环指令

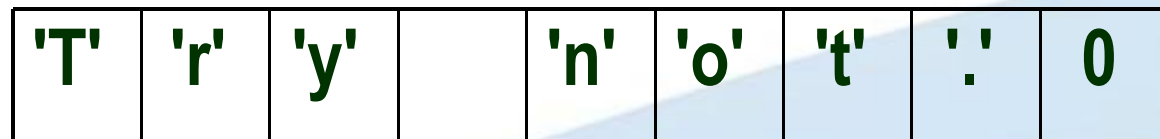
ECX = 字符个数

srcmsg



ESI

dstmsg



EDI

目的字符串显示

```
mov eax,offset dstmsg
```

```
call dispmsg ;显示目的字符串内容
```

操作过程

```
D:\MASM>make32 eg0211
```

```
D:\MASM>eg0211.exe
```

```
Try your best, why not.
```

寄存器间接寻址程序

;数据段

srcmsg byte 'Try your best, why not.',0
dstmsg byte sizeof srcmsg dup(?)

;代码段

mov ecx,lengthof srcmsg
mov esi,offset srcmsg
mov edi,offset dstmsg
again: mov al, [esi]
mov [edi], al

add esi,1
add edi,1
loop again
mov eax,offset dstmsg
call dispmsg

寄存器间接寻址案例：指针解引用

C 语言程序

```
1 int value = 100;
2 int *p = &value;
3 int main() {
4     int x;
5     x = *p;
6     return 0;
7 }
```

Assembly 程序

```
1 .data
2 value DWORD 100
3 p      DWORD ?
4 x      DWORD ?
5
6 .code
7 main PROC
8     mov ebx, OFFSET value
9     mov p, ebx
10    mov ebx, p
11    mov edx, [ebx]
12    mov x, edx
13    ret
14 main ENDP
15
```

理解：p 中保存地址，*p 表示根据地址取值；因此案例中必须出现 `mov edx, [ebx]`。

本讲总结

- 存储器的寄存器间接寻址
 - ▶ 有效地址通过寄存器提供
- 方便对数组的元素或字符串的字符进行操作
 - ▶ 数组(字符串)的首（尾）地址赋值给寄存器
 - ▶ 增减寄存器值指向不同的元素(字符)
- 寄存器间接寻址的操作数没有类型
 - ▶ 其类型由另一个操作数的寄存器或变量类型决定
 - ▶ 若另一个操作数也无类型（立即数），需显式说明

汇编语言程序设计

3.3 存储器的寄存器相对寻址



存储器的寻址方式

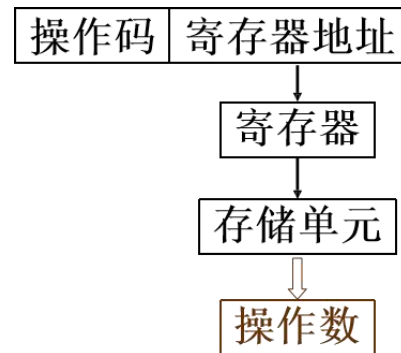
1. 存储器的直接寻址：有效地址只有位移偏移量部分，且只包含在指令代码中，就是存储器的直接寻址方式。直接寻址用于存取变量。例如，将变量COUNT的内容传给ECX的指令：

`mov ecx, count`;也可以表达`mov ecx[count]`

2. 寄存器间接寻址：有效地址放在寄存器中。例如：

`mov edx, [ebx]`;双字传送，EBX间接寻址主存数据段；

如果EBX=405000H, 那么该指令等价于 `mov edx, DS:[405000H]`



3. 寄存器相对寻址：寄存器相对寻址的有效地址是寄存器的内容与位移量之和。例如：

`mov esi, [ebx+4]`;源操作数可以表达为 `[4][ebx]`, 或者 `4[ebx]`

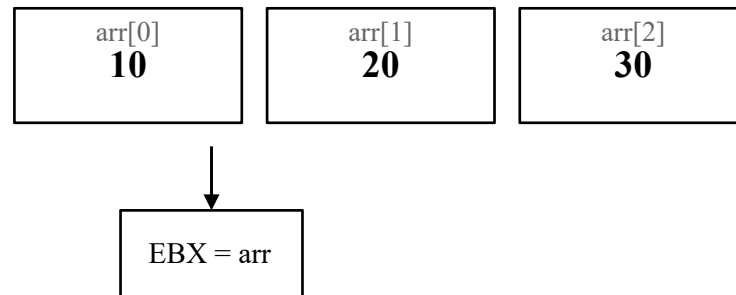
偏移量还可以是其他常量，以及变量所表示的地址，例如 `mov eax, count[esi]`



寄存器相对寻址

- 有效地址 = 寄存器内容 + 位移量。
- 典型指令：mov esi, [ebx+4]
- 等价写法还可以写成 4[ebx] 或 [4][ebx]。
- 常用于数组元素、结构体成员等“基址 + 偏移”的访问。

数组示意



mov esi, [ebx+4]

第 2 个 int 的地址 = 首地址 + 4 字节

寄存器相对寻址案例：数组元素访问

C 语言程序

```
1 int arr[3] = {10, 20, 30};
2 int main() {
3     int x;
4     x = arr[1];
5     return 0;
6 }
```

Assembly 程序

```
1 .data
2 arr DWORD 10, 20, 30
3 x     DWORD ?
4
5 .code
6 main PROC
7     mov ebx, OFFSET arr
8     mov esi, [ebx+4]
9     mov x, esi
10    ret
11 main ENDP
12 END main
```

理解：arr 的首地址放入 EBX，访问 arr[1] 时偏移 4 字节，所以案例中必须出现 mov esi, [ebx+4]。

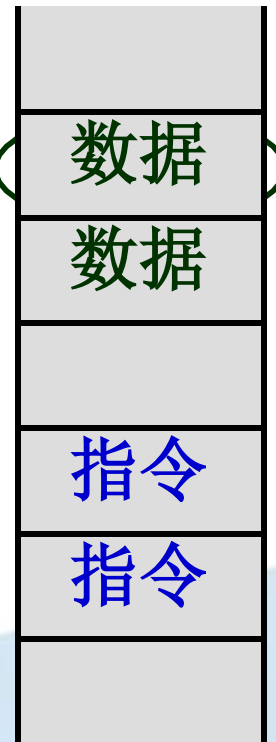
操作数在那儿！

数据来自主存储器→存储器寻址

00405000H

00401000H

地址

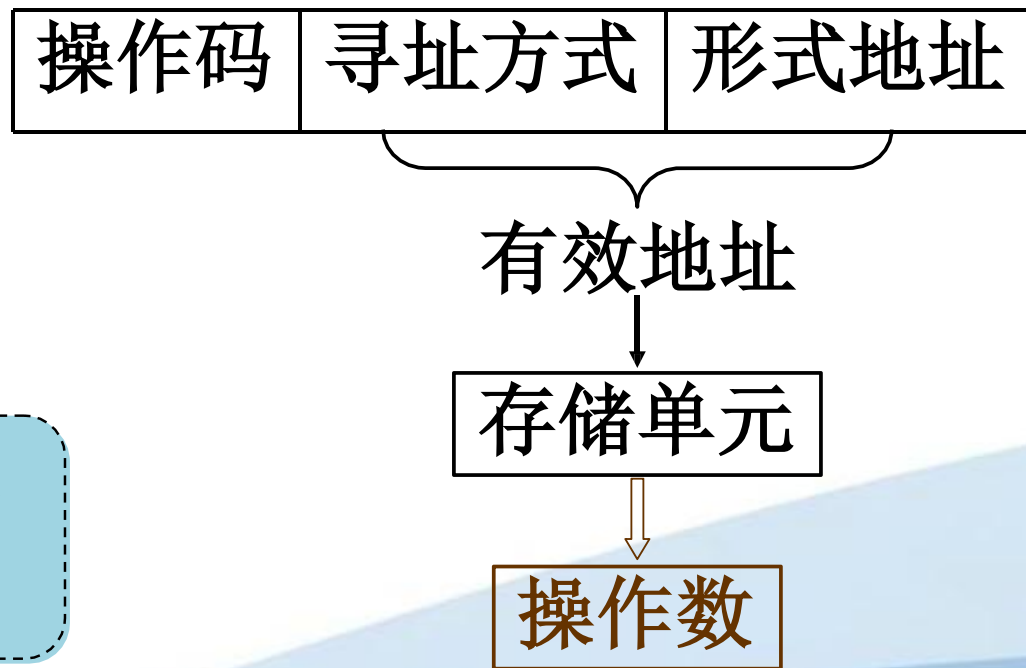


存储器寻址

➤ 操作数在主存中，通过存储器地址指示

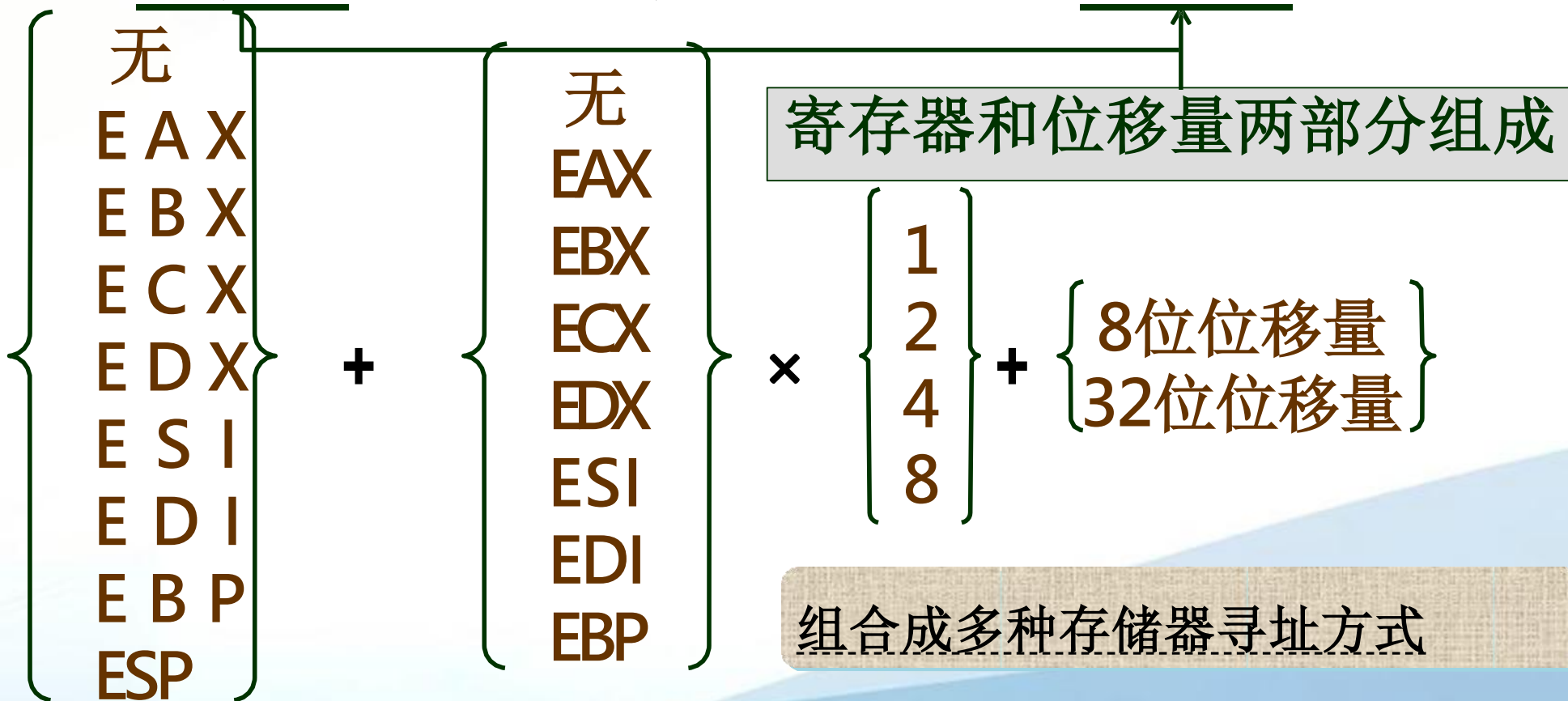
- ▶ 指令代码表达形式地址
- ▶ 由形式地址结合规则 经过计算得到有效地址EA (Effective Address)

处理器将有效地址转换为物理地址访问存储单元



32位存储器寻址

基址寄存器 + 变址寄存器 × 比例 + 位移量



寄存器相对寻址

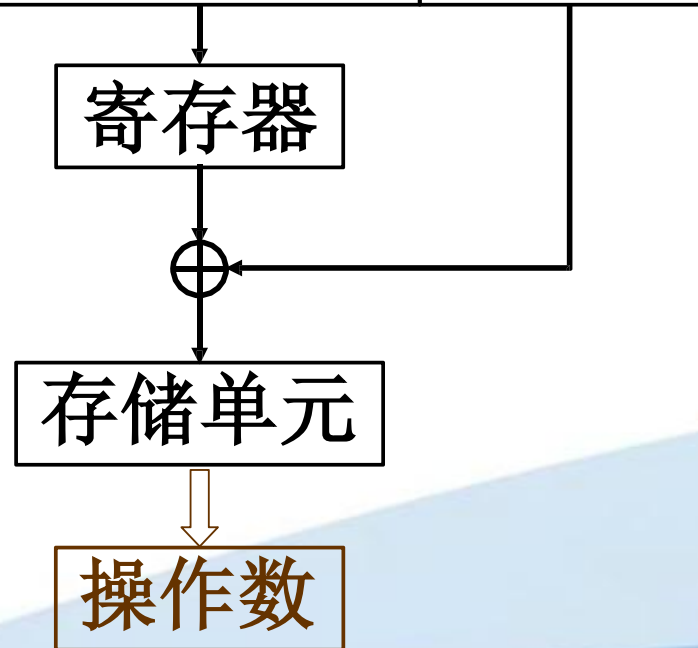
➤ 有效地址是寄存器内容与位移量之和

▶ 寄存器要用中括号括起

操作码	寄存器地址	位移量
-----	-------	-----

`mov esi, [ebx+4]` ;位移量: 4
`mov edi, [ebp-08h]` ;位移量: -08H
`mov eax, count[esi]` ;位移量: count

主存储器采用字节编址 地址的加减
是以主存字节单元为单位



寄存器相对寻址的多种表达形式

`mov esi, [ebx+4]` $\longrightarrow$ `4[ebx]` `[4][ebx]`

;位移量: 4, 无类型

`mov edi, [ebp-08h]` $\longrightarrow$ `[-8h][ebp]`

;位移量: -08H, 无类型

`mov eax, count[esi]` $\longrightarrow$ `[esi+count]` `[count][esi]`

;位移量: `count`, `count`定义的类型

;表示变量所在的偏移地址用作相对寻址的位移量

字符串复制

;数据段

srcmsg byte 'Try your best, why not.',0

dstmsg byte sizeof srcmsg dup(?)

srcmsg

'T'	'r'	'y'		'n'	'o'	't'	'.'	0
-----	-----	-----	--	-----	-----	-----	-----	---

dstmsg

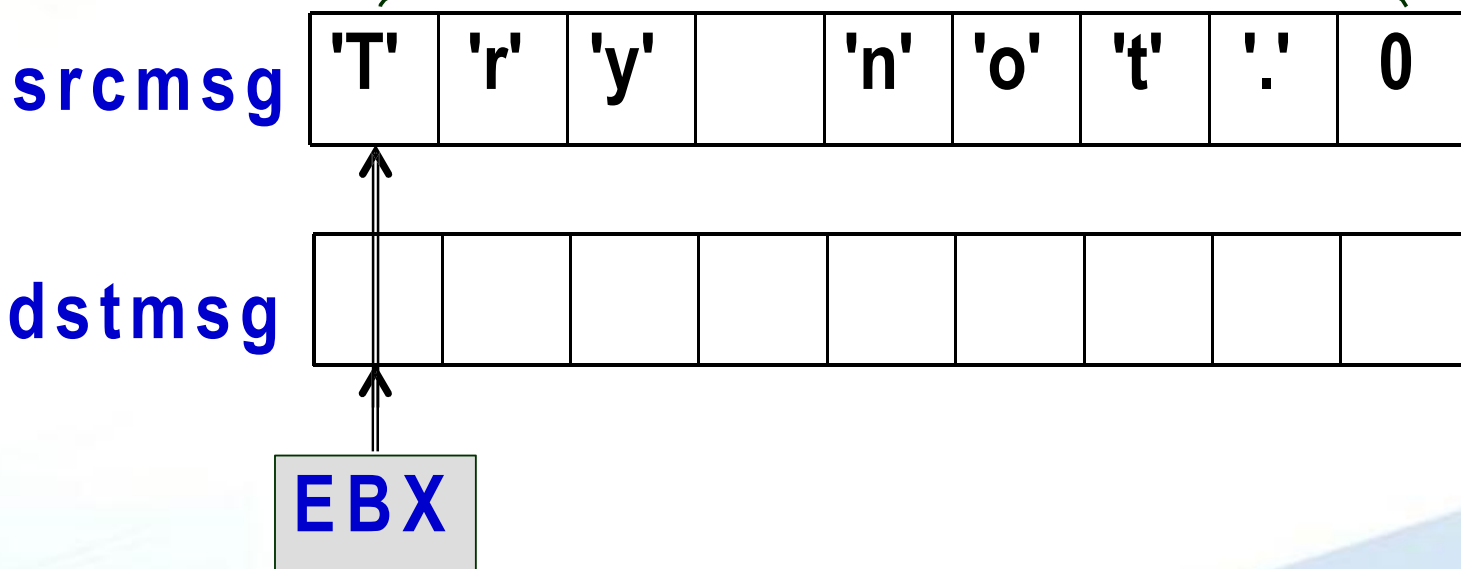
--	--	--	--	--	--	--	--	--

寄存器指向字符位置

```
mov ecx,lengthof srcmsg
```

```
mov ebx,0
```

ECX = 字符个数



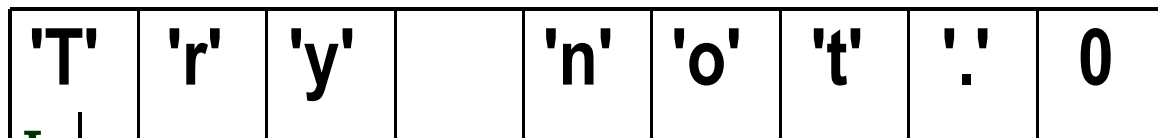


字符传送

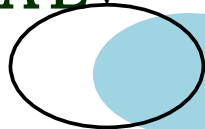
```
mov al, srcmsg[ebx]
```

```
mov dstmsg[ebx], al; 取源串一个字符送AL  
; 将AL传送给目的串位置
```

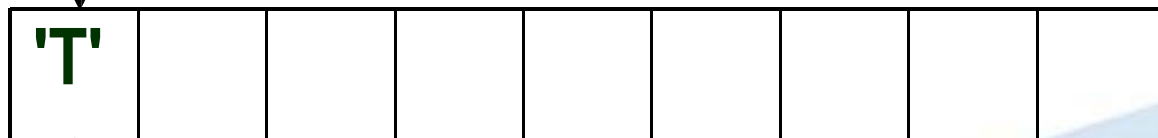
srcmsg



AL



dstmsg



EBX

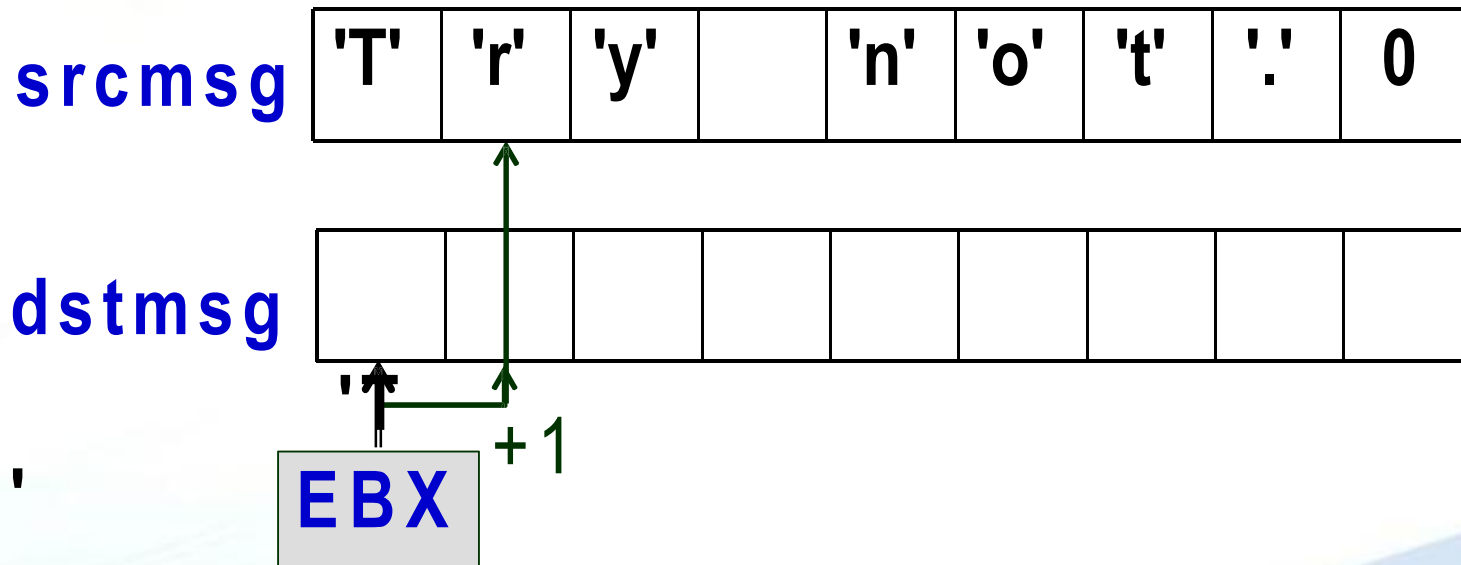


指向下一个字符

add ebx,1

;加1

ADD是加法指令



字符串复制：逐个字符传送

again: **mov al, srcmsg[ebx]**

;传送一个字符

;指向下一个字符

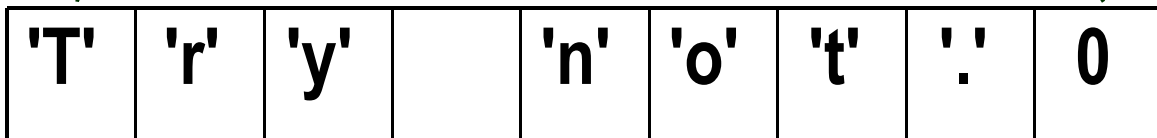
loop again

;字符个数减1，不为0继续

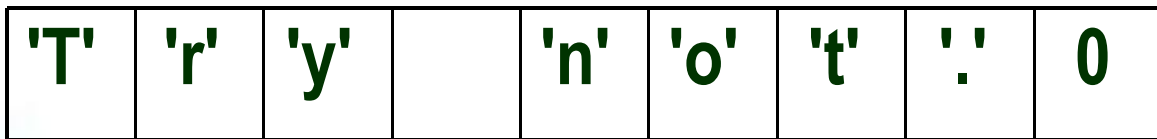
LOOP是循环指令

ECX = 字符个数

srcmsg



dstmsg



EBX

目的字符串显示

```
mov eax,offset dstmsg
```

```
call dispmsg ;显示目的字符串内容
```

操作过程

```
D:\MASM>make32 eg0212
```

```
D:\MASM>eg0212.exe
```

```
Try your best, why not.
```

寄存器相对寻址案例 A：常量偏移

C 语言程序

```
1 int arr[3] = {10, 20, 30};
2 int main() {
3     int x;
4     x = arr[1];
5     return 0;
6 }
```

Assembly 程序

```
1 .data
2 arr DWORD 10, 20, 30
3 x    DWORD ?
4
5 .code
6 main PROC
7     mov ebx, OFFSET arr
8     mov esi, [ebx+4]
9     mov x, esi
10    ret
11 main ENDP
12 END main
```

解释

EBX 保存数组首地址；访问 arr[1] 时，偏移量为 4 字节。
因此关键语句是 mov esi, [ebx+4]。

寄存器相对寻址案例 B：符号偏移

C 语言程序

```
1 int count[4] = {10, 20, 30, 40};
2 int main() {
3     int i = 2;
4     int x;
5     x = count[i];
6     return 0;
7 }
```

Assembly 程序

```
1 .data
2 count DWORD 10, 20, 30, 40
3 x      DWORD ?
4
5 .code
6 main PROC
7     mov esi, 8
8     mov eax, count[esi]
9     mov x, eax
10    ret
11 main ENDP
12 END main
```

解释

这里让 ESI 直接保存字节偏移量 8。
count[esi] 表示：从 count 的首地址出发，再偏移 ESI 字节。

寄存器相对寻址程序

;数据段

```
srcmsg byte 'Try your best, why not.',0  
dstmsg byte sizeof srcmsg dup(?)
```

;代码段

```
mov ecx,lengthof srcmsg  
mov ebx,0  
again: mov al, srcmsg[ebx]  
mov dstmsg[ebx], al
```

```
add ebx,1  
loop again  
mov eax,offset dstmsg  
call dispmsg
```

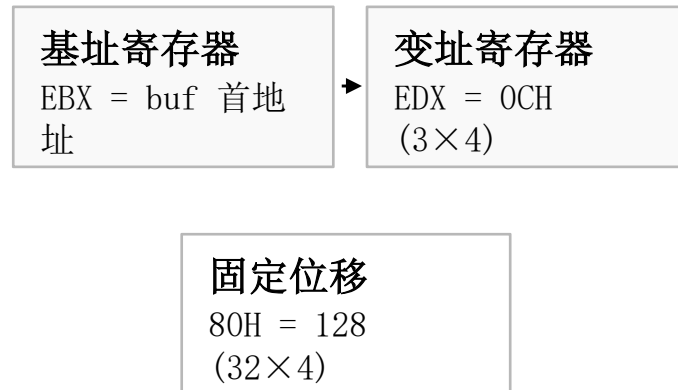
扩展学习 变址寻址

- 有效地址由基址寄存器、变址寄存器和位移量共同形成。
- 适合处理查表、数组块偏移、记录块访问等情况。
- 这里变址寄存器中常常已经保存“字节偏移量”。

mov eax, [ebx+edx+80h]

EA = EBX + EDX + 80H

地址形成示意



$$\mathbf{EA = EBX + EDX + 80H}$$

含义分解

EBX: 基址
EDX: 变化部分
80H: 固定偏移

变址寻址：C / 汇编对照案例

C 语言程序

```
1 int buf[100];
2 int main() {
3     int i = 3;
4     int x;
5     x = buf[32 + i];
6     return 0;
7 }
```

Assembly 程序

```
1 .data
2 buf DWORD 100 DUP(?)
3 x     DWORD ?
4
5 .code
6 main PROC
7     mov ebx, OFFSET buf
8     mov edx, 0Ch
9     mov eax, [ebx+edx+80h]
10    mov x, eax
11    ret
12 main ENDP
13 END main
```

解释

$EDX = 0CH = 12 = 3 \times 4$; $80H = 128 = 32 \times 4$ 。

因此 $[ebx+edx+80h]$ 访问的是 $buf[32+3]$ ，也就是 $buf[35]$ 。



变址寻址小结

- 地址 = 基址 + 变址 + 位移。
- 常用于数组块、查找表、记录块中的元素访问。
- 变址寄存器中常放“已经换算好的字节偏移量”。

与相对寻址的区别

相对寻址通常是“一个寄存器 + 一个偏移”。
变址寻址则多了一个变化寄存器。

一句话

除了固定偏移，还要再加一个动态偏移。

带比例的变址寻址

- 有效地址由基址、变址、比例因子和位移量共同形成。
- 比例因子用于把“数组下标”自动换算成字节偏移。
- 比例因子常见取值为 1、2、4、8。

mov eax, [ebx+esi*4-80h]

EA = EBX + ESI × 4 - 80H

地址形成示意

基址寄存器

EBX = arr 首地址

变址寄存器

ESI = i
这里取 35

比例因子

×4
int 为 4 字节

位移量

-80H
= -128

$$\mathbf{EA = EBX + ESI \times 4 - 80H}$$

关键理解

ESI 可以直接放数组下标；
×4 自动完成“下标 → 字节偏移”的换算。

带比例的变址寻址：C / 汇编对照案例

C 语言程序

```
1 int arr[100];
2 int main() {
3     int i = 35;
4     int x;
5     x = arr[i - 32];
6     return 0;
7 }
```

Assembly 程序

```
1 .data
2 arr DWORD 100 DUP(?)
3 x     DWORD ?
4
5 .code
6 main PROC
7     mov ebx, OFFSET arr
8     mov esi, 35
9     mov eax, [ebx+esi*4-80h]
10    mov x, eax
11    ret
12 main ENDP
13 END main
```

解释

ESI=35, $35 \times 4 = 140$; 80H=128, 所以总偏移是 12。

$12 = 3 \times 4$, 因此最终访问的是 arr[3], 也就是 arr[i-32]。



带比例的变址寻址小结

- 地址 = 基址 + 下标 $\times$ 元素大小 + 位移。
- 最适合表达数组下标访问。
- 看到 $*4$ 时，应立刻想到“每个元素占 4 字节”。

记忆要点

比例因子负责“自动乘上元素大小”。
所以它比普通变址寻址更贴近数组下标的写法。

常见比例

- 1: 字节
- 2: 字
- 4: 双字 / int
- 8: 四字 / long long

全部寻址方式总对比

寻址方式	典型语句	关键含义	C 场景	记忆点
寄存器寻址	<code>mov ecx, eax</code>	寄存器中的数据传送	中间结果	无 []
立即数寻址	<code>add eax, 5</code>	常量写在指令中	常量赋值	数直接给出
直接寻址	<code>mov ecx, count</code>	变量名给出地址	普通变量	名字就能定位
间接寻址	<code>mov edx, [ebx]</code>	按寄存器地址取值	指针 *p	寄存器里装地址
相对寻址	<code>mov esi, [ebx+4]</code>	基址 + 常量偏移	<code>arr[1]</code>	偏移按字节
相对寻址	<code>mov eax, count[esi]</code>	符号地址 + 寄存器偏移	<code>count[i]</code>	基址 + 位移
变址寻址	<code>mov eax, [ebx+edx+80h]</code>	基址 + 变址 + 位移	<code>buf[32+i]</code>	多一个动态偏移
比例变址	<code>mov eax, [ebx+esi*4-80h]</code>	基址 + 下标 × 大小 + 位移	<code>arr[i-32]</code>	自动乘元素大小



课堂练习：判断寻址方式

请判断下列语句属于哪种寻址方式

题目

```
1 mov eax, 5
2 mov eax, ebx
3 mov eax, count
4 mov eax, [ebx]
5 mov eax, [ebx+8]
6 mov eax, count[esi]
7 mov eax, [ebx+edx+80h]
8 mov eax, [ebx+esi*4-80h]
```

参考答案提示

1. 立即数寻址
2. 寄存器寻址
3. 直接寻址
4. 寄存器间接寻址
5. 寄存器相对寻址
6. 寄存器相对寻址
7. 变址寻址
8. 带比例的变址寻址

看懂一条汇编指令，先看操作数从哪里来，再看地址是怎样算出来的。

- 寄存器寻址：数据就在寄存器中。
- 立即数寻址：常量直接写在指令里。
- 存储器寻址：关键在于有效地址 EA 的形成。
- 从直接、间接、相对到变址与比例变址，地址形成越来越灵活。

最后一句话

不同寻址方式的差别，不在于 mov 本身，而在于“操作数在哪里”以及“地址怎么来”。

可继续衔接

段寄存器
比例变址的更多形式
数组循环与字符串处理

本讲总结

- 存储器的寄存器相对寻址
 - ▶ 有效地址通过寄存器内容与位移量相加获得
- 方便对数组的元素或字符串的字符进行操作
 - ▶ 数组(字符串)的首地址作为位移量
 - ▶ 寄存器赋值0，或者元素(字符)个数
 - ▶ 增减寄存器值指向不同的元素(字符)
- 位移量是数字常量，寄存器相对寻址的操作数没有类型
 - ▶ 位移量用变量名表达，则具有变量的类型

1. 了解《九章算术》如何计算两个数的最大公约数和最小公倍数求法。
2. 画出求两个数最大公约数的流程图，写出c语言代码。
3. 画出求两个数最小公倍数的流程图，写出c语言代码。

说明：要求提供流程图和C语言代码，手写/电子截图均可。